

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
print(tf.__version__)
```

2.19.0

```
fashion_mnist = tf.keras.datasets.fashion_mnist
```

```
(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()
```

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>

29515/29515 _____ 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>

26421880/26421880 _____ 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>

5148/5148 _____ 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>

4422102/4422102 _____ 0s 0us/step

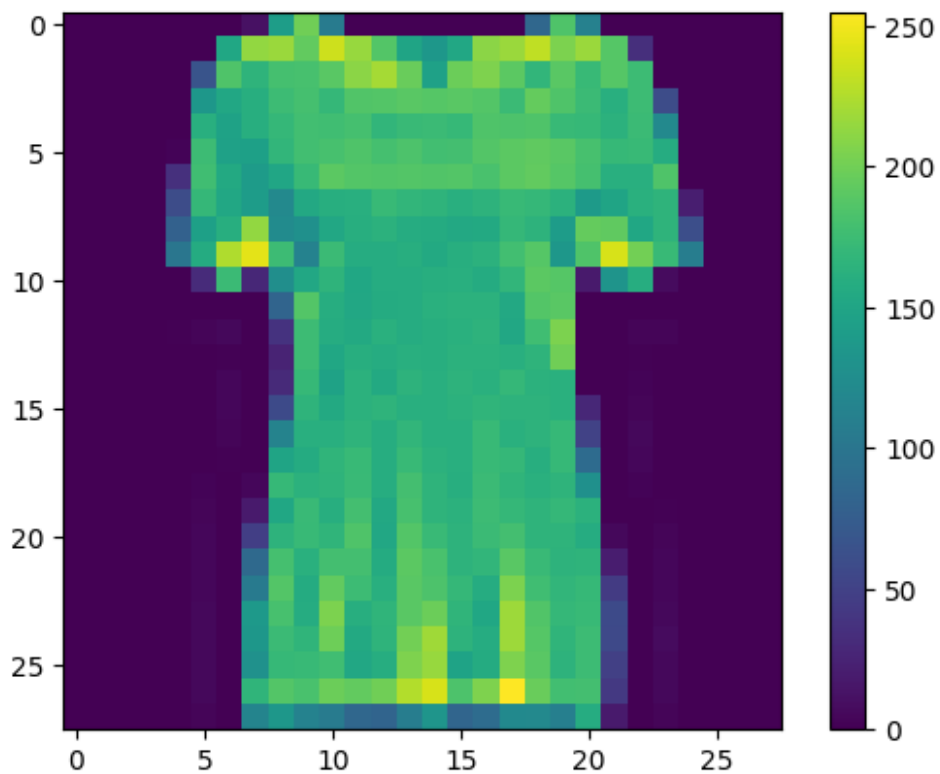
```
train_images.shape
```

(60000, 28, 28)

```
test_images.shape
```

(10000, 28, 28)

```
plt.figure()
plt.imshow(train_images[10])
plt.colorbar()
plt.grid(False)
plt.show()
```



```
train_images=train_images/255.0
test_images=test_images/255.0

plt.figure(figsize=(10,10))
for i in range(20):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```



```

train_images = train_images.reshape(train_images.shape[0], 28, 28,
1).astype('float32')
test_images = test_images.reshape(test_images.shape[0], 28, 28,
1).astype('float32')

model=tf.keras.Sequential([
    tf.keras.layers.Conv2D(64,
(3,3),activation='relu',input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Conv2D(64,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    #tf.keras.layers.Dense(128,activation=
tf.keras.layers.LeakyReLU(alpha=0.3)),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(64,activation='relu'),
    tf.keras.layers.Dropout(0.5),

```

```

    tf.keras.layers.Dense(10,activation='softmax'),
])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
    super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

model.compile(optimizer='rmsprop',

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
            metrics=['accuracy'])

```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type) Param #	Output Shape	
conv2d (Conv2D) 640	(None, 26, 26, 64)	
max_pooling2d (MaxPooling2D) 0	(None, 13, 13, 64)	
conv2d_1 (Conv2D) 36,928	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D) 0	(None, 5, 5, 64)	
flatten (Flatten) 0	(None, 1600)	
dense (Dense) 204,928	(None, 128)	

0	dropout (Dropout)	(None, 128)	
8,256	dense_1 (Dense)	(None, 64)	
0	dropout_1 (Dropout)	(None, 64)	
650	dense_2 (Dense)	(None, 10)	

Total params: 251,402 (982.04 KB)

Trainable params: 251,402 (982.04 KB)

Non-trainable params: 0 (0.00 B)

model.fit(train_images, train_labels, epochs=5)

Epoch 1/5

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/numpy.py:717: UserWarning: "`sparse\_categorical\_crossentropy` received `from\_logits=True`, but the `output` argument was produced by a Softmax activation and thus does not represent logits. Was this intended?

output, from_logits = _get_logits(

1875/1875 ————— 11s 3ms/step - accuracy: 0.6587 - loss: 0.9764

Epoch 2/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8527 - loss: 0.4451

Epoch 3/5

1875/1875 ————— 7s 4ms/step - accuracy: 0.8678 - loss: 0.4032

Epoch 4/5

1875/1875 ————— 6s 3ms/step - accuracy: 0.8710 - loss: 0.4003

Epoch 5/5

1875/1875 ————— 7s 3ms/step - accuracy: 0.8727 - loss: 0.4064

<keras.src.callbacks.history.History at 0x7bcde9a91ee0>

```
test_loss, test_acc = model.evaluate(test_images, test_labels,  
verbose=1)
```

```
print('\nTest accuracy:', test_acc)
```

```
313/313 ————— 3s 6ms/step - accuracy: 0.8859 - loss:  
0.3814
```

```
Test accuracy: 0.8859000205993652
```